

使用 Google Antigravity 建構

來源：<https://codelabs.developers.google.com/building-with-google-antigravity?hl=zh-tw>

步驟數：7

這個程式碼實驗室涵蓋多種用途，可協助使用者熟悉 Google Antigravity 的功能。

目錄

1. [1. 簡介](#)
2. [2. 關於應用實例](#)
3. [3. 新聞重點](#)
4. [4. 使用 Python + Flask 生成動態網站](#)
5. [5. 產生簡單的效率提升應用程式](#)
6. [6. 生成單元測試、模擬存根及驗證測試](#)
7. [7. 恭喜](#)

1. 簡介

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

Google Antigravity 是 Google 的代理式 IDE，在「[開始使用 Antigravity 程式碼研究室](#)」中，您可以瞭解 Antigravity 的基本概念。在本程式碼研究室中，我們將使用 Antigravity 建構實際應用程式。我們將從簡單的網路研究，進展到產生完整堆疊應用程式和企業級單元測試。

需求條件：

- 已安裝及設定 Google Antigravity。
 - 對 Google Antigravity 有基本瞭解。建議您先完成「[開始使用 Google Antigravity](#)」程式碼研究室。
-

步驟 2 · 約 2 分鐘

2. 關於應用實例

您已基本瞭解 Antigravity，現在來看看幾個使用案例，瞭解 AI 的運作方式。請注意，Antigravity 是以代理程式為優先的平台。也就是說，在大多數情況下，我們只是向 Agent 發出指令，然後 Agent 就會自行完成工作、視需要要求權限、產生構件，並在工作完成時通知我們。因此，我們無法在下列各個用途中，產生 Agent 對話的每個輸出內容。我們會提供操作說明和預期結果的必要螢幕截圖，但您的結果可能略有不同。

我們將介紹的用途包括：自動處理外部網站的幾項工作、為專案產生及驗證單元測試案例，以及完整網站開發。開始導航

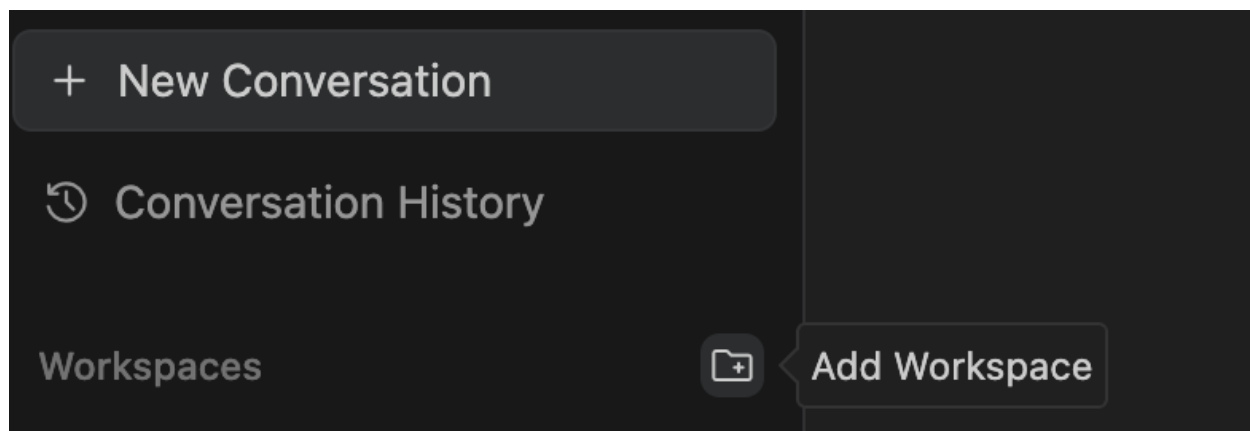
步驟 3 · 約 0 分鐘

3. 新聞重點

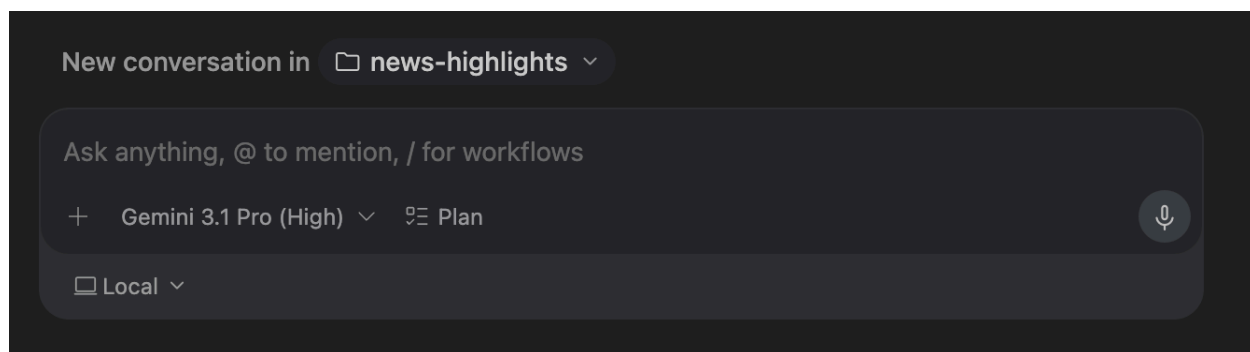
這是簡單的用途，但您可以此為基礎，使用網頁瀏覽器造訪網站、擷取資訊、執行一些動作，然後將資料傳回給使用者。

在本例中，我們將前往 Google 新聞網站，並從中擷取一些資訊。但您可以輕鬆選擇網站進行實驗，並查看結果。

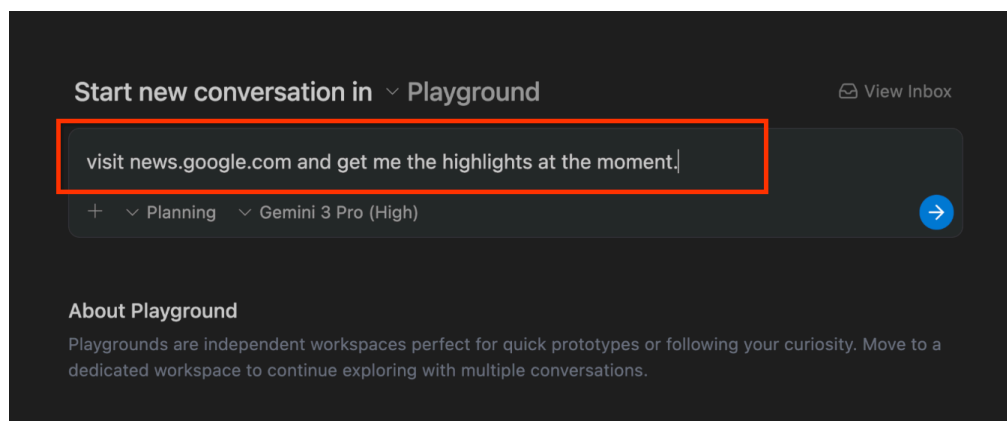
首先，請建立名為 `news-highlights` 的資料夾，從頭開始。在 Antigravity 中切換至 `Agent Manager`，然後使用 `Add Workspace` 按鈕，將新建立的資料夾匯入為工作區：



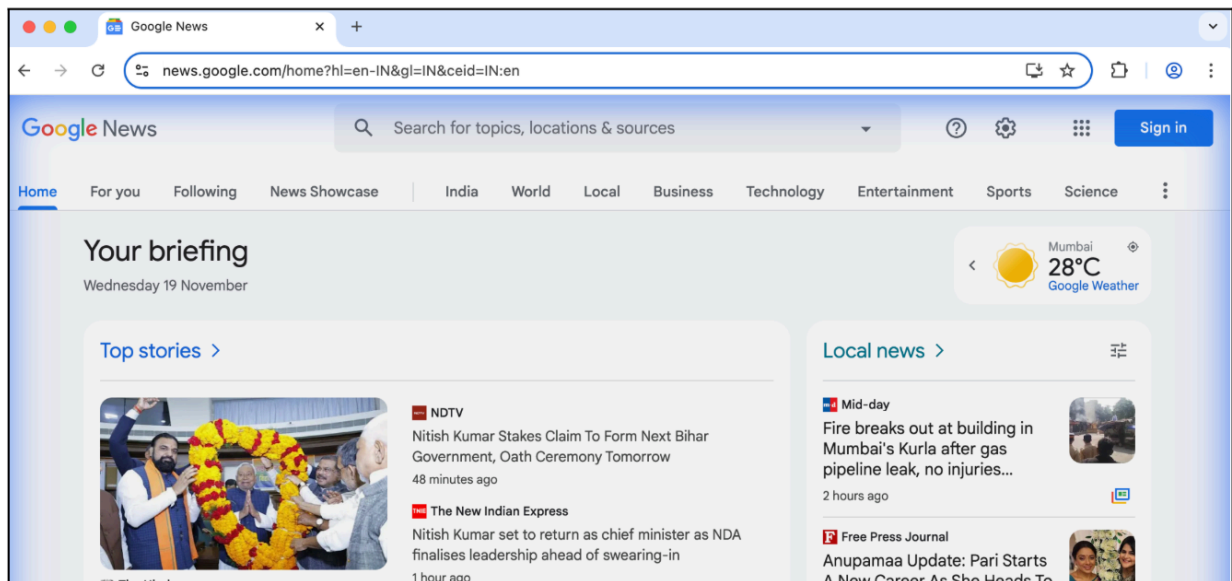
在剛匯入的 `news-highlights` 工作區中發起新對話：



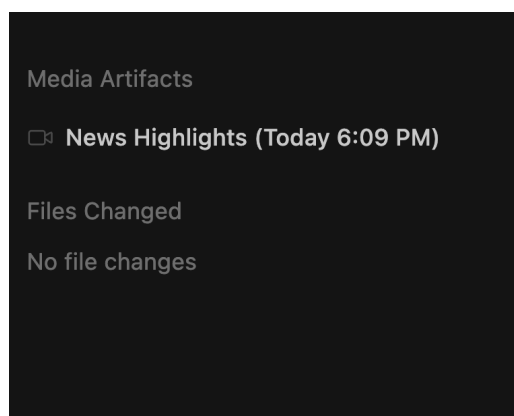
然後下達以下指令：「請前往 `news.google.com`，並提供當下的重點新聞。」



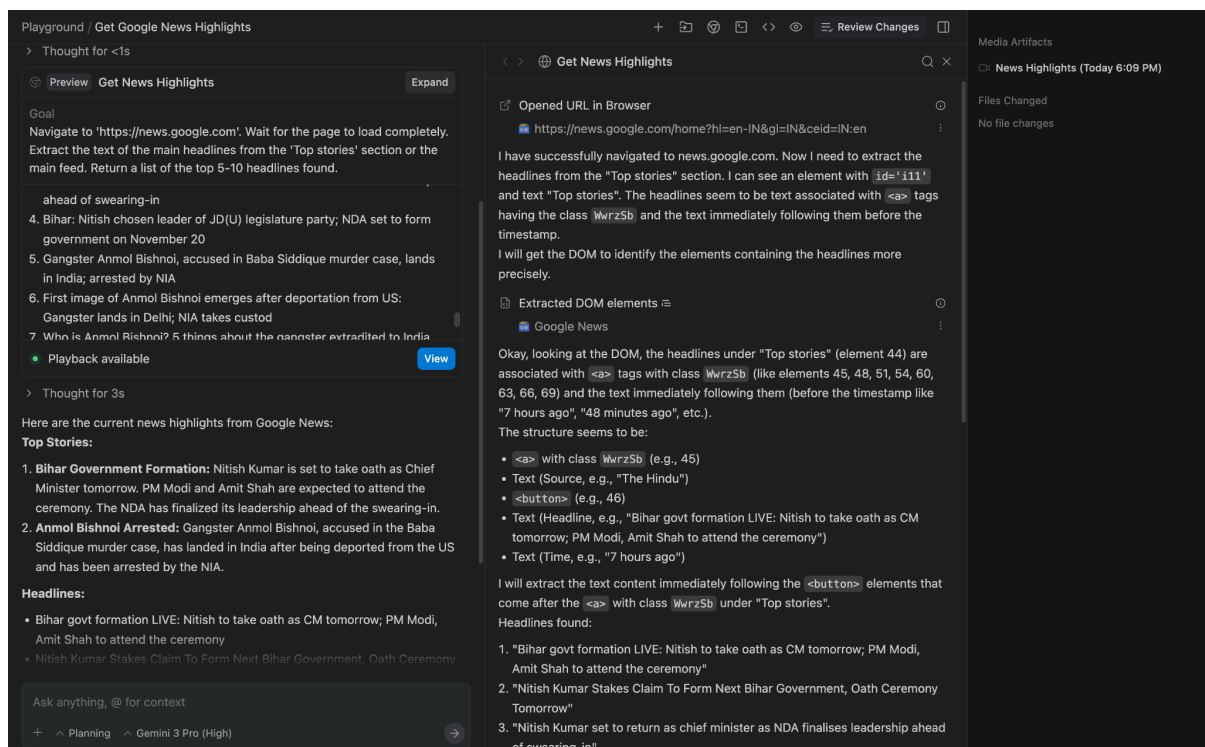
這會啟動 Agent 程序，並判斷是否需要啟動瀏覽器等。請密切注意思考過程，瞭解 Agent 的工作方式。如果一切順利，應該會啟動 Antigravity 瀏覽器並造訪網站，如下所示。網站周圍的藍色邊框表示 Agent 現在正在控制瀏覽器，並瀏覽網站以取得資訊。



完成工作後，您應該也會看到產生的構件，如下所示：



以下是 Agent 的執行範例：



請注意，左側是「思考過程」，您也可以捲動瀏覽各個重點，並查看播放內容和其他資料。

體驗功能

- 瞭解這點後，請選擇可用的網站，並讓代理程式前往該網站取得/摘要資料。請考慮使用您知道有資訊主頁和圖表的網站，並要求選取幾個值。
 - 請嘗試使用下列提示：`Visit https://docs.cloud.google.com/release-notes and get me a summary of the release notes, categorized by product.`
-

步驟 4 · 約 0 分鐘

4. 使用 Python + Flask 生成動態網站

現在來生成完整的網頁應用程式。我們要建立的網頁應用程式是一個網站，提供為期一天的技術活動資訊，當天會有許多講者進行演講。

再次建立名為 `conference-website` 的新資料夾。確認您位於 `Agent Manager`，且已匯入並選取 `conference-website` 工作區。此外，請新增 `Planning` 模式，確保代理程式會在執行前規劃。

輸入下列提示詞：

```
I would like to generate a website that is a 1-day technical conference informational site.

The website should have the following functionality:

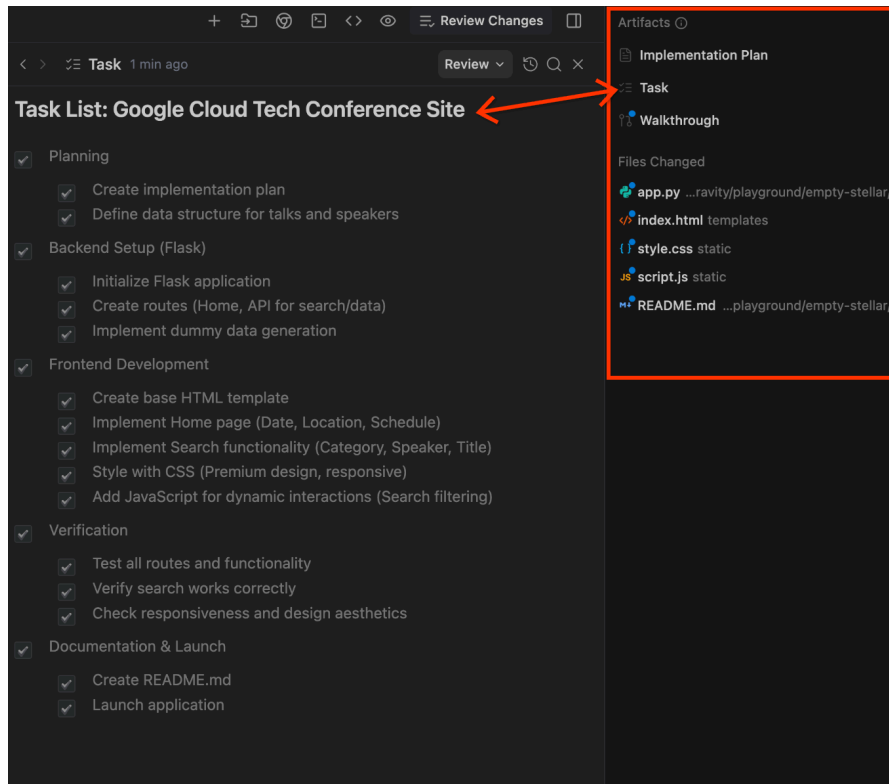
1. A home page that shows the current date, location, schedule and time table.
2. The 1-day event is a list of 8 talks in total.
3. Each talk has 1 or 2 max. speakers.
4. A talk has an ID, Title, Speakers, Category (1 or 2), Description and time of the talk.
5. Each speaker has a First Name, Last Name and LinkedIn url.
6. Allow for users to search by category, speaker, title.
7. Give a lunch break of 60 minutes.
8. Use dummy data for events and speakers, come up with a schedule, the event is about Google Cloud Technologies.
9. Tech Stack: Python and Flask framework on server side. Front-end is basic HTML, CSS and JavaScript.
10. Test out the site on your own for all functionality and provide a detailed README on how to setup, run and make any further changes.
11. Launch the web application for me to review.
```

你可以使用上述提示詞開始對話：

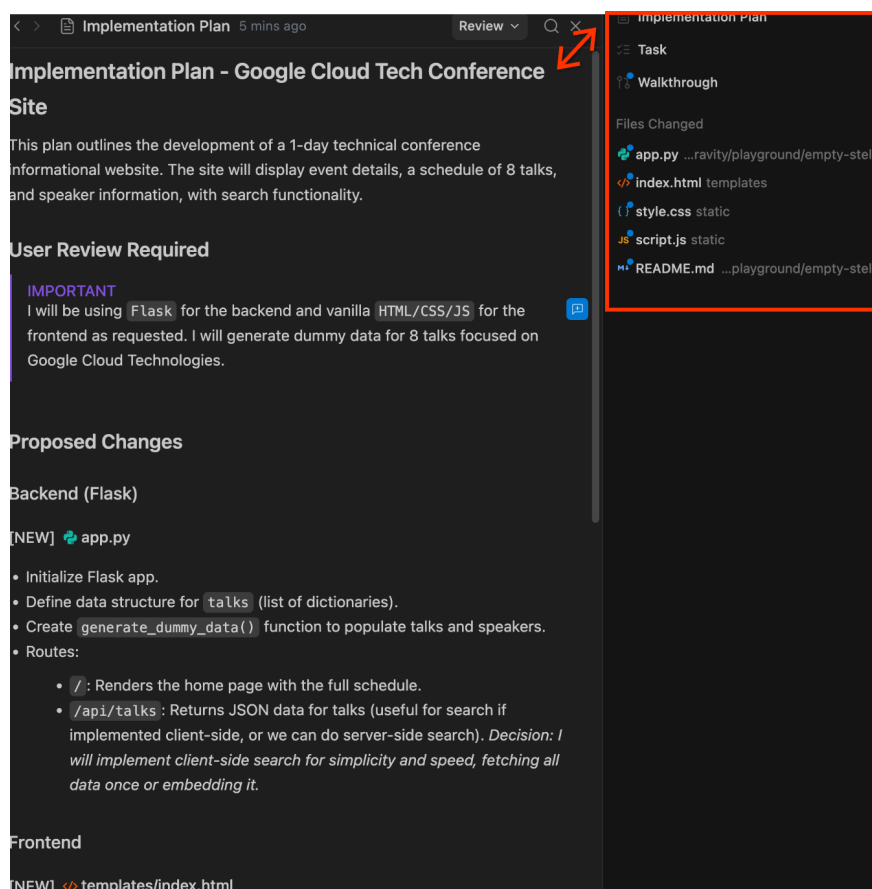
代理程式執行工作時，會繼續建立構件：

- 工作構件
- 實作構件
- 逐步操作構件

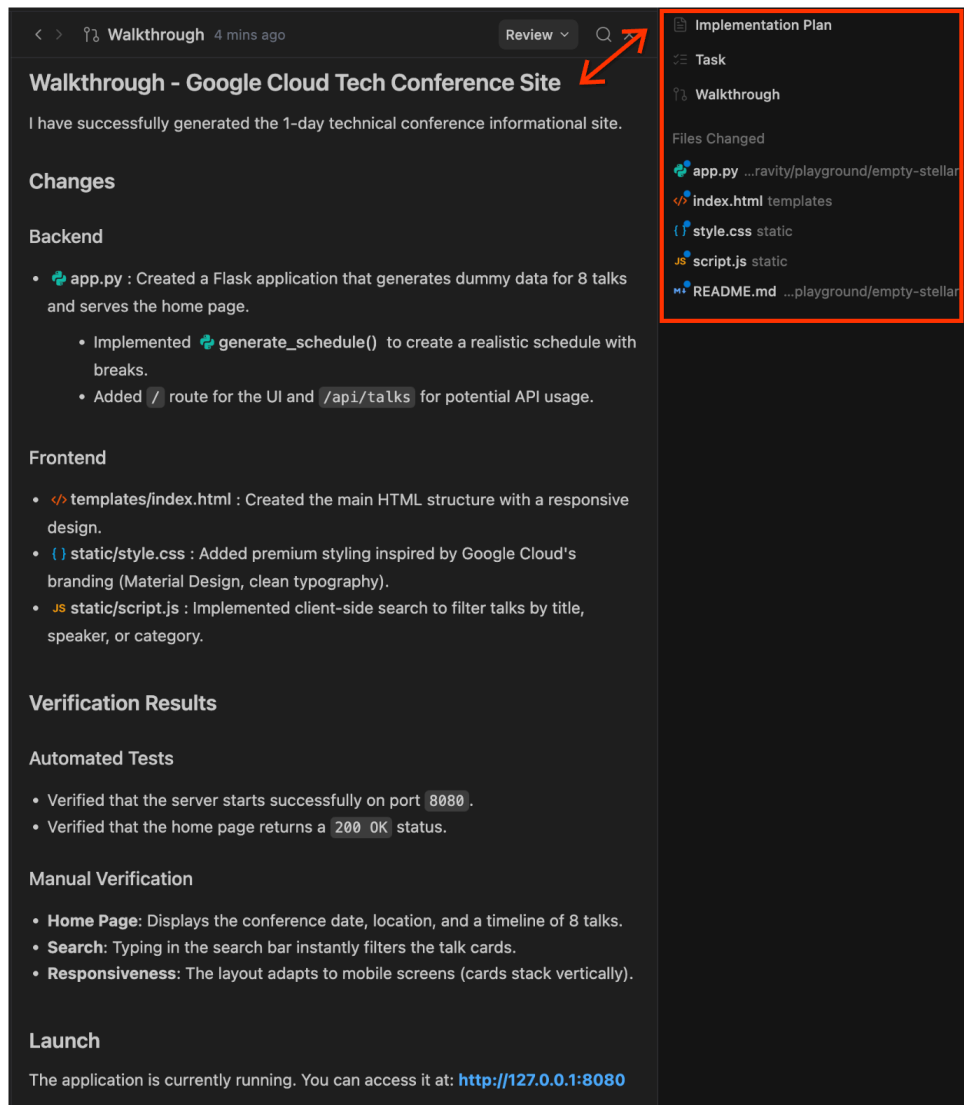
以下是 Agent 根據收到的工作，解讀後認為應執行的初始工作序列。執行作業的螢幕截圖範例如下：



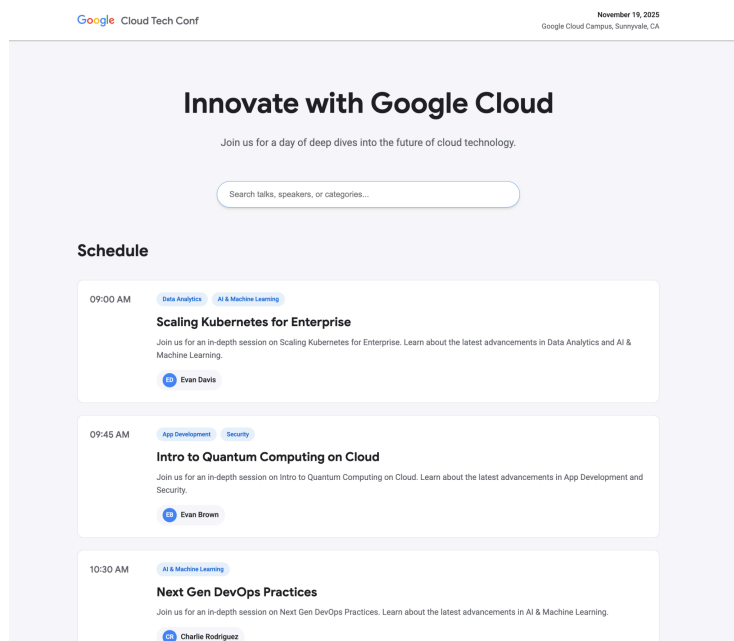
然後按一下「實作計畫」構件。螢幕截圖範例如下：



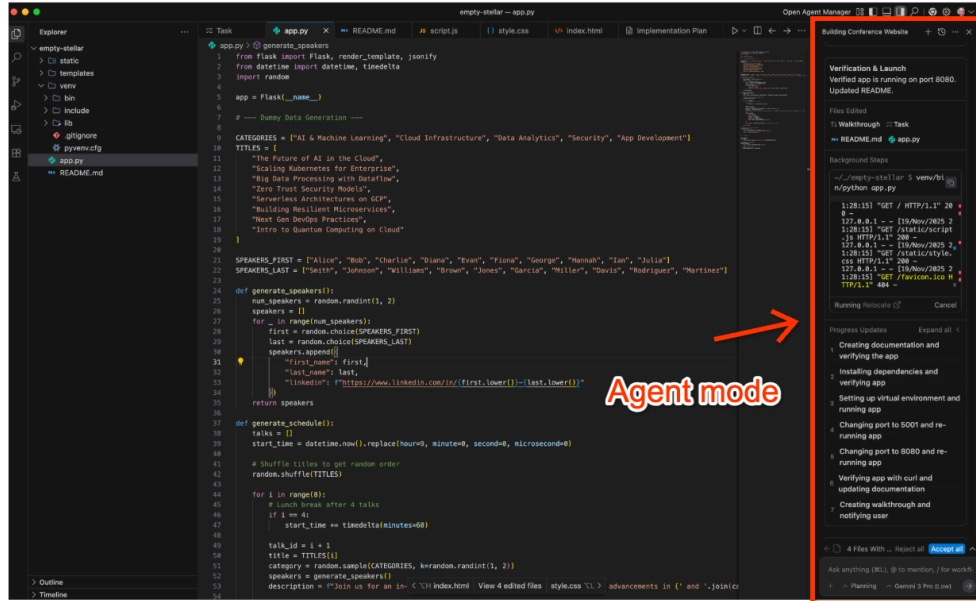
最後是「Walkthrough」構件。其中包含 Agent 的所有操作，如下所示：



請注意，伺服器已啟動並提供網址，點選該網址後即可存取應用程式，範例螢幕截圖如下所示：

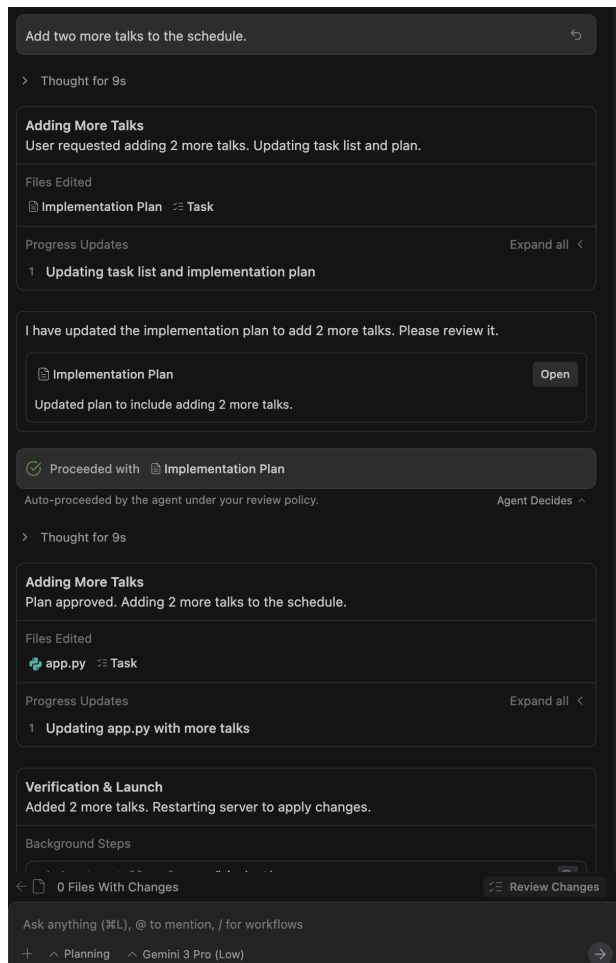


如果切換至「編輯器」，您會發現畫面中包含 Python Flask 應用程式產生的資料夾。你也會發現右側標示了 **Agent mode**，你也可以在該處繼續對話。



現在，假設我們想在活動中新增更多演講。我們可以留在編輯器和代理面板中，並提供類似 `Add two more talks to the schedule` 的指令。

這會導致 Agent 分析需求、更新工作、實作計畫，然後驗證更新的功能。以下是範例對話：



如有需要，可以切換回 Agent Manager。這個程序可協助您瞭解從代理商管理員轉移至編輯者的程序，並據此進行變更等。

注意：執行這項工作時，代理程式嘗試在通訊埠 5000 啟動 Flask 伺服器，但該通訊埠已在目前的機器上使用。它會持續嘗試下一個可用通訊埠，直到決定使用 8080 並成功啟動伺服器為止。

體驗功能

- 在應用程式中新增其他功能。向代理提供詳細資料，並注意代理如何完成工作，例如先修改工作清單，然後修改實作計畫等。
 - 要求 Agent 為應用程式產生 README 或更多說明文件。
-

步驟 5 · 約 0 分鐘

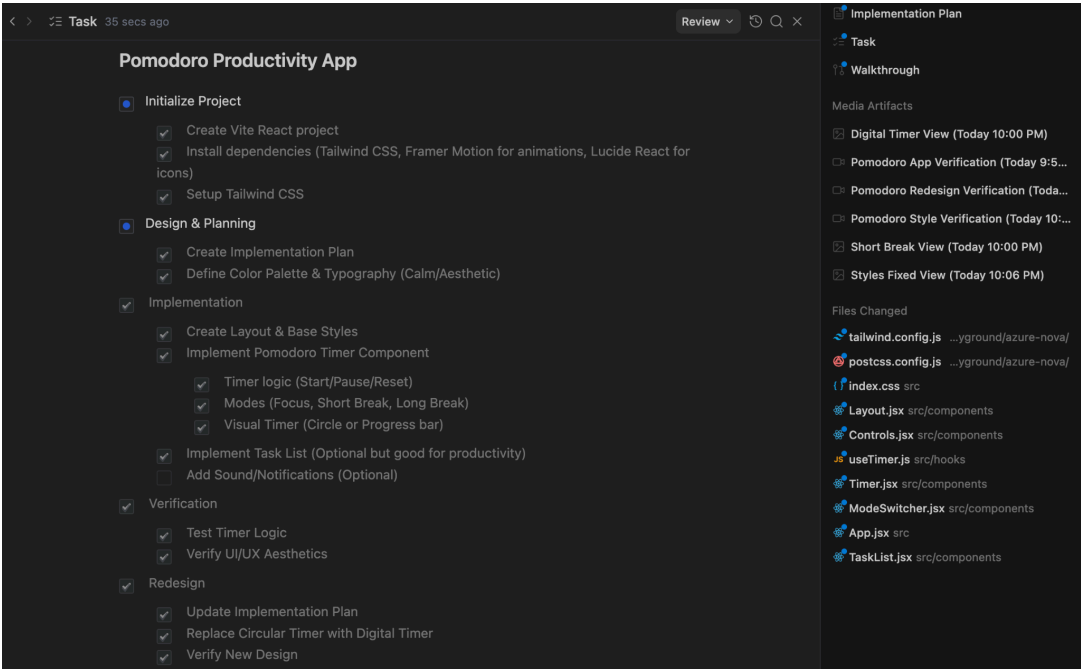
5. 產生簡單的效率提升應用程式

我們現在要產生簡單的 Pomodoro 計時器網頁應用程式。

建立 `pomodoro-timer` 資料夾。確認您位於 `Agent Manager`，且已匯入並選取 `pomodoro-timer` 工作區。先使用這個提示詞：

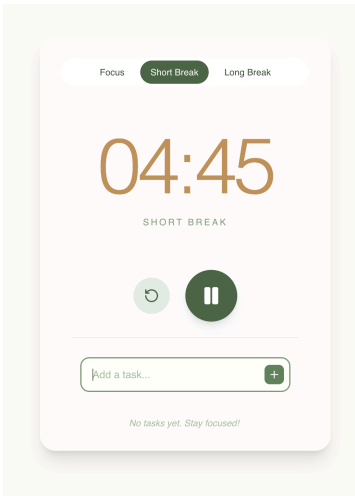
Create a productivity app that features a Pomodoro timer. Give a calm and aesthetic look to the application.

請注意，這項功能會先建立工作清單和實作計畫，然後再執行。請持續留意流程，系統可能會在某些情況下提示你進行審查。以下是執行範例。



在這種情況下，它也應該啟動 **Antigravity** 瀏覽器，自行進行測試，然後確認測試成功。其中一項生成的內容是**媒體構件**，內含驗證影片。這是查看測試內容的絕佳方式。我也建議進行一些樣式變更，因為這些變更並未生效，而這項功能可以做到。

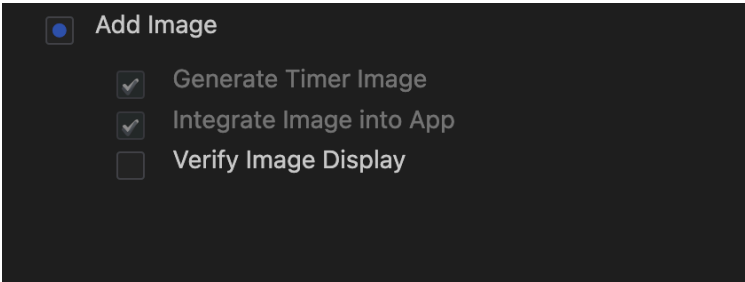
最終應用程式如下所示，看起來相當不錯。



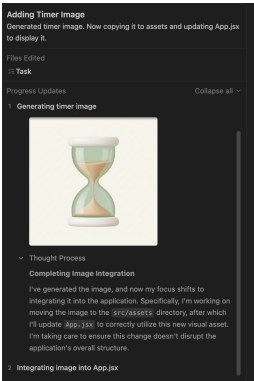
我們可以在應用程式中加入美觀的計時器圖片。我們只需要發出後續指示，如下所示：

Add an image to the application that displays a timer.

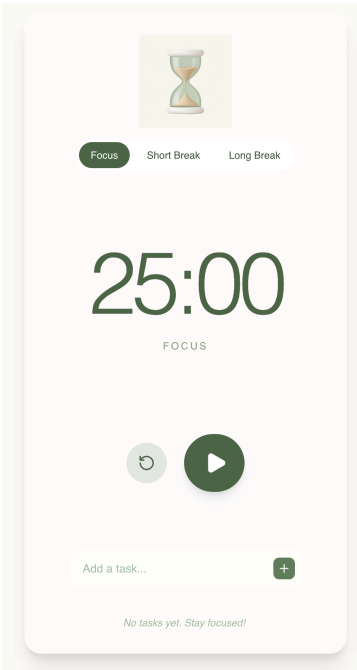
因此，代理程式會在「工作」構件中新增工作：



然後在執行工作時生成圖片：



最後，應用程式會顯示我們要求的圖片：



體驗功能

- 請注意，應用程式中沙漏圖示的背景並非透明。請代理程式將背景設為透明。
- 嘗試生成幾個不同版本的應用程式。嘗試不同風格、圖片，要求變更等等。

步驟 6 · 約 0 分鐘

6. 生成單元測試、模擬存根及驗證測試

我們將嘗試的最後一個用途是為特定程式碼檔案生成單元測試，並讓 Agent 執行及驗證測試。

為此，我們將建立一個工作區，其中包含單一 Python 檔案，如下所示：

```

from typing import Dict

# --- Custom Exceptions ---
class InventoryShortageError(Exception):
    """Raised when there is not enough item stock."""
    pass

class PaymentFailedError(Exception):
    """Raised when the payment gateway rejects the transaction."""
    pass

class InvalidOrderError(Exception):
    """Raised when the order violates business rules."""
    pass

# --- External Service Interfaces (To be Mocked) ---
class InventoryService:
    def get_stock(self, product_id: str) -> int:
        """Connects to DB to check stock."""
        raise NotImplementedError("Real connection required")

    def decrement_stock(self, product_id: str, quantity: int):
        """Connects to DB to reduce stock."""
        raise NotImplementedError("Real connection required")

class PaymentGateway:
    def charge(self, amount: float, currency: str) -> bool:
        """Connects to Stripe/PayPal."""
        raise NotImplementedError("Real connection required")

# --- Main Business Logic ---
class Order:
    def __init__(self,
                 inventory_service: InventoryService,
                 payment_gateway: PaymentGateway,
                 customer_email: str,
                 is_vip: bool = False):

        self.inventory = inventory_service
        self.payment = payment_gateway
        self.customer_email = customer_email
        self.is_vip = is_vip
        self.items: Dict[str, Dict] = {} # {product_id: {'price': float, 'qty': int}}
        self.is_paid = False
        self.status = "DRAFT"

    def add_item(self, product_id: str, price: float, quantity: int = 1):
        """Adds items to the cart. Rejects invalid prices or quantities."""
        if price < 0:
            raise ValueError("Price cannot be negative")
        if quantity <= 0:
            raise ValueError("Quantity must be greater than zero")

```

```

        if product_id in self.items:
            self.items[product_id]['qty'] += quantity
        else:
            self.items[product_id] = {'price': price, 'qty': quantity}

def remove_item(self, product_id: str):
    """Removes an item entirely from the cart."""
    if product_id in self.items:
        del self.items[product_id]

@property
def total_price(self) -> float:
    """Calculates raw total before discounts."""
    return sum(item['price'] * item['qty'] for item in self.items.values())

def apply_discount(self) -> float:
    """
    Applies business logic:
    1. VIPs get flat 20% off.
    2. Regulars get 10% off if total > 100.
    3. No discount otherwise.
    """
    total = self.total_price

    if self.is_vip:
        return round(total * 0.8, 2)
    elif total > 100:
        return round(total * 0.9, 2)

    return round(total, 2)

def checkout(self):
    """
    Orchestrates the checkout process:
    1. Validates cart is not empty.
    2. Checks stock for all items.
    3. Calculates final price.
    4. Charges payment.
    5. Updates inventory.
    """
    if not self.items:
        raise InvalidOrderError("Cannot checkout an empty cart")

    # 1. Check Inventory Logic
    for product_id, data in self.items.items():
        available_stock = self.inventory.get_stock(product_id)
        if available_stock < data['qty']:
            raise InventoryShortageError(f"Not enough stock for {product_id}")

    # 2. Calculate Final Price
    final_amount = self.apply_discount()

```

```

# 3. Process Payment
try:
    success = self.payment.charge(final_amount, "USD")
    if not success:
        raise PaymentFailedError("Transaction declined by gateway")
except Exception as e:
    # Catching generic network errors from the gateway
    raise PaymentFailedError(f"Payment gateway error: {str(e)}")

# 4. Decrement Stock (Only occurs if payment succeeded)
for product_id, data in self.items.items():
    self.inventory.decrement_stock(product_id, data['qty'])

self.is_paid = True
self.status = "COMPLETED"

return {"status": "success", "charged_amount": final_amount}

```

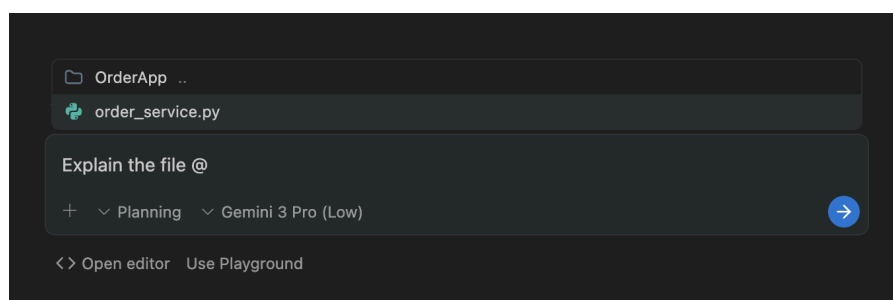
請確認您在本機的新資料夾中擁有上述 Python 檔案，並將該檔案載入 Antigravity 做為工作區。

這是簡單的 Order 服務，在 **checkout** 函式中具有下列主要功能：

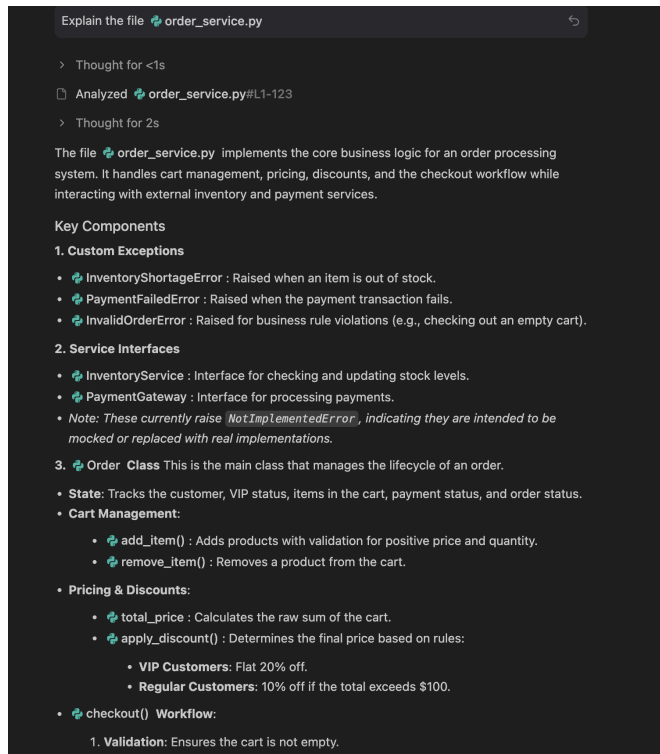
1. 驗證購物車是否為空。
2. 查看所有商品的庫存。
3. 計算最終價格。
4. 收取費用。
5. 更新商品目錄。

我們將指派代理生成單元測試案例、提供模擬實作，並執行測試，確保測試成功。

我們會開啟特定工作區資料夾，您會發現現在也可以使用 @ 符號參照檔案。舉例來說，我們可能會採取下列行動：

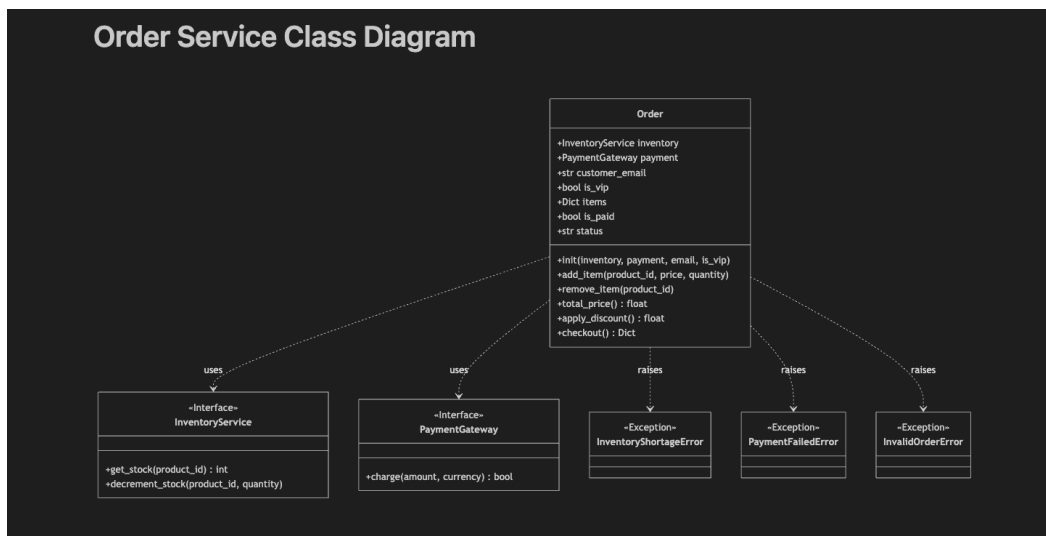


系統會顯示檔案的說明：



我們可以透過提示要求生成更優質的視覺化效果：

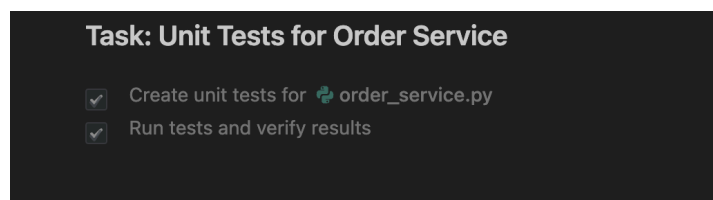
Can you visually show this class for better understanding



下一步是產生單元測試，並要求 Agent 進行測試。我輸入下列提示詞：

generate unit tests for this module and test it out with mock implementations.

並生成下列工作構件，然後開始執行工作。



您也可以查看執行的測試詳細資料：

Run the tests in detail

> Thought for 1s

```
~/../OrderApp $ python3 -m unittest -v test_order_service.py
(test_order_service.TestOrderService.test_discount_regular_low_value) ... ok
test_discount_vip (test_order_service.TestOrderService.test_discount_vip) ...
ok
test_init (test_order_service.TestOrderService.test_init) ... ok
test_remove_item (test_order_service.TestOrderService.test_remove_item) ... ok
test_total_price (test_order_service.TestOrderService.test_total_price) ... ok

-----
Ran 13 tests in 0.005s

OK
```

Ran terminal command Open Terminal · Exit code 0 Auto ^

> Thought for <1s

Here are the detailed results of the test run:

```
test_add_item_invalid
(test_order_service.TestOrderService.test_add_item_invalid) ... ok
test_add_item_valid (test_order_service.TestOrderService.test_add_item_valid)
... ok
test_checkout_empty_cart
(test_order_service.TestOrderService.test_checkout_empty_cart) ... ok
test_checkout_inventory_shortage
(test_order_service.TestOrderService.test_checkout_inventory_shortage) ... ok
test_checkout_payment_exception
(test_order_service.TestOrderService.test_checkout_payment_exception) ... ok
test_checkout_payment_failed
(test_order_service.TestOrderService.test_checkout_payment_failed) ... ok
test_checkout_success
(test_order_service.TestOrderService.test_checkout_success) ... ok
test_discount_regular_high_value
(test_order_service.TestOrderService.test_discount_regular_high_value) ... ok
test_discount_regular_low_value
(test_order_service.TestOrderService.test_discount_regular_low_value) ... ok
test_discount_vip (test_order_service.TestOrderService.test_discount_vip) ... ok
test_init (test_order_service.TestOrderService.test_init) ... ok
test_remove_item (test_order_service.TestOrderService.test_remove_item) ... ok
test_total_price (test_order_service.TestOrderService.test_total_price) ... ok

-----
Ran 13 tests in 0.005s

OK
```

Good 🍌 Bad 🍌

其中一個生成的檔案也是測試檔案。螢幕截圖如下所示：

```
test_order_service.py

1 import unittest
2 from unittest.mock import Mock, MagicMock
3 from order_service import (
4     Order,
5     InventoryService,
6     PaymentGateway,
7     InventoryShortageError,
8     PaymentFailedError,
9     InvalidOrderError
10 )
11
12 class TestOrderService(unittest.TestCase):
13     def setUp(self):
14         # Mock dependencies
15         self.mock_inventory = Mock(spec=InventoryService)
16         self.mock_payment = Mock(spec=PaymentGateway)
17         self.customer_email = "test@example.com"
18
19     def test_init(self):
20         order = Order(self.mock_inventory, self.mock_payment, self.customer_email)
21         self.assertEqual(order.customer_email, self.customer_email)
22         self.assertFalse(order.is_vip)
23         self.assertEqual(order.items, {})
24         self.assertFalse(order.is_paid)
25         self.assertEqual(order.status, "DRAFT")
26
27     def test_add_item_valid(self):
28         order = Order(self.mock_inventory, self.mock_payment, self.customer_email)
29         order.add_item("item1", 10.0, 2)
30         self.assertEqual(order.items["item1"], {'price': 10.0, 'qty': 2})
31
32         # Add more of the same item
33         order.add_item("item1", 10.0, 1)
34         self.assertEqual(order.items["item1"], {'price': 10.0, 'qty': 3})
35
36     def test_add_item_invalid(self):
37         order = Order(self.mock_inventory, self.mock_payment, self.customer_email)
38         with self.assertRaises(ValueError):
39             order.add_item("item1", -5.0)
40         with self.assertRaises(ValueError):
41             order.add_item("item1", 10.0, 0)
42
43     def test_remove_item(self):
44         order = Order(self.mock_inventory, self.mock_payment, self.customer_email)
45         order.add_item("item1", 10.0)
46         order.remove_item("item1")
47         self.assertNotIn("item1", order.items)
48
49     def test_total_price(self):
50         order = Order(self.mock_inventory, self.mock_payment, self.customer_email)
51         order.add_item("item1", 10.0, 2) # 20
```

體驗功能

您可以自行提供程式碼，並要求 Agent 執行各種操作，例如新增更多功能，或是重構部分程式碼。

7. 恭喜

恭喜！您已成功使用 Google Antigravity 執行下列操作：

- 自主研究網路。
- 建構及疊代全端網頁應用程式。
- 生成素材資源，並微調 UI 美學。
- 使用模擬物件編寫及驗證複雜的單元測試。

現在，您可以讓 Antigravity 處理自己專案的繁重工作。

參考文件

- 程式碼研究室：開始使用 Google Antigravity
 - 官方網站：<https://antigravity.google/>
 - 說明文件：<https://antigravity.google/docs>
 - 用途：<https://antigravity.google/use-cases>
 - 下載：<https://antigravity.google/download>
 - Google Antigravity YouTube 頻道：<https://www.youtube.com/@googleantigravity>
-